Lenguajes de script en cliente: Javascript

Javascript es un lenguaje de programación interpretado por el navegador y creado por Netscape cuya función original era mejorar la funcionalidad de los formularios en los documentos HTML.

Hoy en día es el lenguaje más usado en el lado del cliente de una aplicación web, aportando dinamismo e interactividad con el usuario.

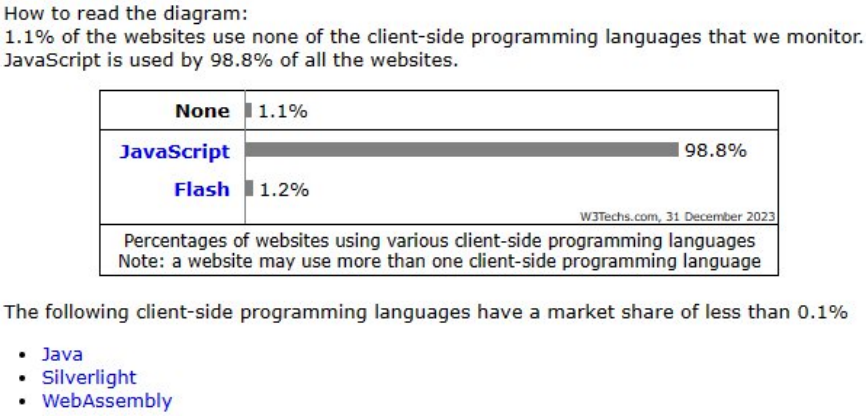


Imagen 1. Fuente https://w3techs.com

Características del lenguaje Javascript

Las principales características de Javascript son las siguientes:

Lenguaje imperativo y estructurado: es un lenguaje imperativo en el que se van ejecutando las sentencias de manera secuencial. Además Javascript permite una programación estructurada construida a base de expresiones (asignaciones a variables, definición de arrays y objetos, invocación a funciones, operadores,...) y sentencias (controles de flujo, declaración de funciones, sentencias de salto,...) o bloques de sentencias.

Lenguaje interpretado y débilmente tipado: Javascript se interpreta en el navegador, así que la apariencia final y ciertos detalles de ejecución dependen de éste (aunque se supone que se siguen unos estándares). Javascript no necesita definir el tipo de una variable a la hora de instanciarla. El tipo de variable se asigna dependiendo del valor que le asignemos a la variable, además si cambiamos el valor que le asignamos, ésta cambiará de tipo de datos.

Cliente y servidor: Javascript se ejecuta en el lado cliente de una aplicación Web, pero también se utiliza para construir aplicaciones de servidor (NodeJS), pero también ha ocupado su sitio en áreas para el desarrollo de APIs de bases de datos como sucede con MongoDB.

Ejecución: El código Javascript se puede ubicar de diferentes formas en la página web y se puede ejecutar de dos formas distintas: durante la carga de la página web o en respuesta a eventos.

Multiplataforma: El hecho de ser un lenguaje interpretado y que se ejecuta tanto en cliente, como en servidor, permite a Javascript tener múltiples interpretes en diferentes navegadores (Google Chorme, Microsoft Edge, Safari, Opera,...) que se ejecutan en diferentes sistemas operativos (Windows, Mac, Linux,...).

Orientado a objetos: Javascript es un lenguaje orientado a objetos, aunque podríamos decir que no es un lenguaje puro de orientación a objetos como lo puede ser C++ o Java ya que carece de algunas características como herencia.

Hola Mundo Javascript

El programa más sencillo escrito en Javascript sería la siguiente versión del ya clásico "Hola Mundo". El código javascript se añade a la pñagina dentro de la etiqueta `<script>`.

Aunque hemos situado el código JavaScript dentro del elemento `<body>` de la página HTML, más adelante veremos qué puede ponerse también dentro de la cabecera. Es decir, dentro del elemento `<head>`.

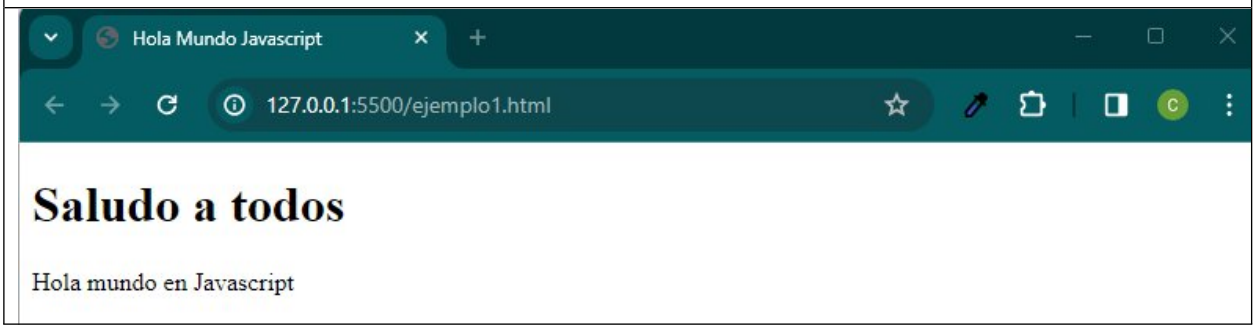
La función que utilizaremos para realizar el Hola Mundo JavaScript será `document.write`. Esta función generará el texto dentro de la página HTML:

```
<!DOCTYPE html>
<html>
<head>
    <title>Hola Mundo Javascript</title>
</head>
<body>
    <script>
        document.write("Hola mundo");
    </script>
</body>
</html>
```

Hola mundo

Si añadimos otros elementos HTML a la página antes y/o después del script, veremos que se muestran de manera secuencial:

```
<!DOCTYPE html>
<html>
    <head>
       <title>Hola Mundo Javascript</title>
    </head>
    <body>
        <h1>Saludo  a todos</h1>
        <script>
            document.write("Hola mundo");
        </script>
        en Javascript
    </body>
</html>
```



Veamos sobre el ejemplo anterior las primeras reglas de sintaxis:

- Los sangrados son importantes para la claridad del código como en cualquier otro lenguaje, pero no son obligatorios como en Python
- Las sentencias se pueden separar con un punto y coma (;) o con un salto de línea. Si escribimos cada instrucción en una línea (la mejor opción por claridad en el código) el ; no es obligatorio pero si recomendado dado que nos permitirá adaptarnos con mayor facilidad a otros lenguajes de programación que sí lo exigen como, por ejemplo, PHP o Java.
- Si queramos encadenar más de una sentencia en la misma línea sí que es obligatorio usar el punto y coma como separador.

Uso de comentarios

Los comentarios pueden escribirse de dos formas diferentes. Cuando ocupan una única línea (ya sea completa o parcialmente) se utiliza el símbolo // y cuando el comentario ocupa varias líneas usamos /* para el inicio y */ para el final:

```
<script>
          //este el el comentario que ocupa una línea
          document.write("Hola mundo");
          /*Con esto comentamos
          varias líneas de
          código*/
</script>
```

MUY IMPORTANTE: todo lo que escribas en el código Javascript se envía al navegador del usuario y éste puede verlo, así que no es ni el sitio ni el modo de dejar nada que pueda comprometer la seguridad de tu aplicación (contraseñas, por ejemplo) o de ofender al usuario o gastar bromas de ningún tipo.

Si mostramos el código fuente de la página, podemos ver los comentarios:

```
<!DOCTYPE html>
<html>
    <head>
<title>Hola Mundo Javascript</title>
    </head>
    <body>
        <h1>Saludo  a todos</h1>
        <script>
            //este el el comentario que ocupa una línea
            document.write("Hola mundo");
            /*Con esto comentamos
            varias líneas de
            código*/
        </script>
        en Javascript
```

Variables

Lo primero que tenemos que saber sobre Javascript es qué es un lenguaje sensible a mayúsculas Así tendremos diferentes variables si las llamamos de la siguiente forma:

```
var variable1;
var Variable1;
var VARIABLE1;
```

En javascript no hace falta declarar las variables antes de usarlas aunque podemos hacerlo si queremos y es una buena práctica. Los siguientes ejemplos son iguales:

```
//Definimos la variable num1 y le asignamos el valor 4
     var num1;
     num1=4;
```

```
//La variable num1 vale 4
      num1=4;
```

```
//Definimos la variable num1 y le asignamos el valor 4
    var num1=4;
```

Como hemos visto en las características de Javascript, éste es lo que se denomina un lenguaje de programación "debilmente tipado", es decir, cuando definimos una variable no tenemos por que indicar qué tipo de dato guardará (enteros, decimales, cadenas de texto, etc.) pero una vez que inicializamos una con un determinado valor, aunque podemos, ya no deberíamos de cambiarla a otro diferente. Esto, además, dificultaría la interpretación del código y no es lo que buscamos.

Para nombrar una variable pueden usarse caracteres alfanuméricos y el símbolo de subrayado (_) pero sólo podrán empezar por una letra o por (_), nunca pueden empezar por un número.

`Contador, curso2, cantidad_puntos, _curso`	✓
`2curso, 2_curso, &inicio`	✗

Ámbito de las variables

El ámbito de las variables (donde tienen existencia) es el bloque donde se declaren o usen por primera vez. Llamamos **variables locales** a las que tienen como ámbito una función o un bloque concreto y **globales** a las que tienen como ámbito cualquier función de una página. Esto se consigue declarándola fuera de todas las funciones (ya las veremos mas adelante). Si existen dos variables con el mismo nombre y diferentes ámbitos siempre se tomará la más cercana. En cualquier caso hay que tener mucho cuidado con esto porque puede que las variables no se destruyan de forma automática fuera del bloque donde tienen existencia y esto podría dar lugar a errores. Lo mejor es no repertir variables con el mismo nombre ni usarlas fuera del ámbito de su existencia.

Desde el año 2015 se introdujo como novedad en Javascript, una nueva forma de declarar las variables con la palabra reservada `let`. Las variables declaradas con `let` no pueden redeclararse (con `var` si) lo cual nos proporciona una seguridad adicional al codifica:

```
<script>
    /*Esto funcionaría correctamente, pero
    puede llegar a producir errores o resultados inesperados*/
var ms="Hola mundo";
var ms=0;
document.write(ms);
</script>
```

```
<script>
    /*Error de sintaxis*/
let ms="Hola mundo";
let ms=0;
document.write(ms);
</script>
```

Adicionalmente, las variables definidas con `var` dentro de un bloque que no sea una función (un if, un for, un while...) pueden usarse fuera de él, sin embargo las definidas con `let` no, lo cual refuerza la idea de ámbito de las mismas:

```
<script>
    if (true) {
        var minumero = 8;
    }

    document.write(minumero); // Mostrará un 8
</script>
```

```
    <script>
        if (true) {
  let minumero = 8;
}

document.write(minumero); // Dará error en consola: minumero is not defined
    </script>
```

También en Javascript podemos definir constantes utilizando la palabra reservada `const`

Las constantes se usan igual que las variables pero tienen que ser declaradas e inicializadas con un valor que luego no podrá alterarse durante la vida de nuestra aplicación:

`<script>`
`const PI=3.14159;`
`PI=5;//no puede hacerse`
`PI=PI+1; //no puede hacerse`
`</script>`
	Uncaught TypeError: Assignment to constant variable.
	at ejemplo2.html:8:7

Las variables y constantes pueden guardar números (enteros o decimales), cadenas de texto (entre comillas simples o dobles) o valores booleanos (true o false). Los valores decimales se separan con punto y no con coma.

Operadores aritméticos

Los operadores aritméticos típicos son `=, +, -, *, /` y `%`:

Operador	Significado
`x + y`	Suma los dos operandos.
`x - y`	Resta los dos operandos.
`x * y`	Multiplica los dos operandos.
`x / y`	Divide los dos operandos.
`x % y`	Ejecuta el módulos de los dos operandos. Es el resto de una división.
`++x`	Incrementa en 1 el operando y devuelve el operando
`x++`	Devuelve el operando e incrementa en 1 el operando.
`--x`	Decrementa el operando y devuelve el operando.
`x--`	Devuelve el operando y decrementa en 1 el operando.

Los operadores + y – se usan también como operadores unarios para determinar el signo de una cantidad: +5, -valor, etc.

El signo + es también útil para otras cosas. Por un lado sirve para concatenar cadenas. `"Hola" + " " + "mundo"` daría como resultado la cadena `"Hola mundo"`. Por otro lado, cuando tenemos caracteres numéricos dentro de una cadena el operador + nos ayuda a "transformarlos" fácilmente en números.

Cuando Javascript hace una operación aritmética devuelve el resultado con todos los decimales posibles. Para "remediar" esto usamos el método `toFixed`. Por ejemplo, si la variable `num` guarda el número 3.14159 la expresión `num.toFixed(2)` nos devolverá ese mismo número con sólo dos decimales, o sea, 3,14

Operadores lógicos

Los operadores lógicos nos sirven para hacer comparaciones y siempre devuelven un valor booleano (true o false). Son: `==, !=, <, <=, >, >=`

Podemos combinar diferentes operaciones lógicas con los operadores:

Operador	Significado
X && Y	Operador `AND`, devuelve `true` si ambos operandos son `true`
X || Y	Operador `OR`, devuelve `true` si alguno de los operandos es `true`
!X	Operador `NOT`, invierte el valor del operando X, si es `true` devolverá `false`, si es `false`, devolverá `true`

Cuando comparamos dos cadenas de texto lo que hacemos en realidad es ver cuál es menor alfabéticamente que la otra. La comparación se hace caracter a caracter. Cuando la cadena combina letras (mayúsculas y minúsculas), números y símbolos se usa la posición del caracter en las tablas ASCII para determinar cual es menor o mayor.

Cuando tenemos dudas sobre el resultado en operaciones complejas podemos usar los paréntesis para agrupar resultados. Esto es válido también cuando usamos operadores aritméticos.

Javascript es un lenguaje "crédulo" por lo que cualquier cosa diferente de 0 o una cadena vacía se evalúa como true.

Javascript tiene, además, el operador lógico === denominado de estricta igualdad que sólo valida como cierto cuando la comparación coincide en valor y tipo:

```
"54"==54 // será true
"54"===54 // será false
```

Sentencias de control

Las estructuras de control son las típicas de cualquier lenguaje de programación y las podemos dividir en estructuras selectivas y estructuras repetitivas.

Estructuras selectivas

Las estructuras selectivas sirven para controlar el flujo de un programa, en función del resultado de una condición haremos unas cosas u otras. En Javascript tenemos dos tipos de estructuras selectivas:

- if-else
- switch

if-else

La estructura de control if, evalúa una expresión lógica (condición), si se cumple, es decir tiene el resultado true, se ejecutará el `bloque_sentencias_1`, si no se cumple, se ejecturará el `bloque_sentencias_2`:

```
if (condicion)
{
  bloque_sentencias_1
}
else
{
  bloque_sentencias_2
}
```

La parte del `else` no es obligatoria:

```
if (condicion)
{
  bloque_sentencias_1
}
```

Las llaves sólo son obligatorias si necesitamos ejecutar más de una instrucción en el bloque. Podemos anidar `if` hasta cualquier nivel. Los sangrados y las llaves son en este caso indispensables para que el código sea legible:

```
if(condicion)
{
   if(otra_condicion)
   {
    Bloque_sentencias_1
   }
   else
   {
    Bloque_sentencias_2
   }
}
```

```
if (condicion)
{
  bloque_sentencias_1
}
else if (condicion2)
{
  bloque_sentencias_2
}
else if (condicion3)
{
 bloque_sentencias_3
}
...
else
{
  bloque_sentencias_N1
}
```

Ejemplo:

```
let num1 = 5;
let num2 = 8;
if (num1 > num2) {
  document.write(num1 + " es mayor que " + num2);
}
else if (num1 < num2) {
  document.write(num2 + " es mayor que " + num1);
}
else {
 document.write("Los dos números son iguales");
     }
```

Los valores que son asumidos como `false` serán:

- false
- undefined
- null
- 0
- NaN
- Cadena vacía “”

switch

La estructura selectiva `switch` permite evaluar una expresión contra diferentes valores. En el caso de que la expresión concuerde con un valor, se ejecutará su bloque de sentencias asociado.

En el caso de que haya dos valores que concuerden con el resultado de la expresión, se ejecutarán los bloques de sentencias asociados a dichos valores.

Si no se encuentra un valor asociado a la expresión se ejecutará el bloque de sentencias asociado a la etiqueta `default`.

El formato de la estructura selectiva es el siguiente:

```
switch (expresion) {
  case label1:
    bloque_sentencias_1;
    [break;]
  case label2:
    bloque_sentencias_2;
    [break;]
  ...
  default:
    bloque_sentencias_default;
    [break;]
}
```

Para que solo se ejecute un bloque debemos de utilizar la sentencia `break` al final de cada bloque. De esta forma, cuando encuentre la primera etiqueta que coincida con el valor de la expresión, ejecutará el bloque y se saldrá de la estructura `switch`.

Aunque se suele poner la etiqueta `default en` la última posición, ésta puede aparecer en cualquier posición de la estructura `switch`.

Ejemplo: conocer el día de la semana que es hoy

```
let fecha = new Date();
let dia = fecha.getDay();

        switch (dia) {
            case 1:
                document.write('Hoy es lunes');
                break;
            case 2:
                document.write('Hoy es martes');
                break;
            case 3:
                document.write('Hoy es miércoles');
                break;
            case 4:
                document.write('Hoy es jueves');
                break;
            case 5:
                document.write('Hoy es viernes');
                break;
            case 6:
                document.write('Hoy es sábado');
                break;
            case 7:
                document.write('Hoy es domingo');
                break;
            default:
                document.write('El valor introducido no corresponde con un día');
        }
```

Sentencias repetitivas

Las estructuras repetitivas son aquellas que nos permiten realizar una misma acción un número determinado de veces. El número de veces puede ser un contador o la evaluación de una condición.

Existen varias estructuras repetitivas:

- for
- while
- do..while

Y algunas sentencias que nos ayudan dentro de las estructuras repetitivas:

- break
- continue
- etiquetas

La sentencia for

La sentencia `for` se usa cuando queremos ejecutar un mismo bloque de instrucciones un número determinado (o no) de veces:

<table>
<tr><td>

```
for (i = 0; i < 10; i++)
{
// bloque de sentencias
}
```

</td><td rowspan="2">Inicialización: solo se ejecuta la primera vez que se entra en la estructura.
Condición: condición que debe evaluarse para ejecutar el bloque de sentencias
Actualización: una vez realizado el bloque de sentencias, se incrementa el valor del contador</td></tr>
<tr><td>

```
for (i = 0,j=5; i < j; i++)
{
// bloque de sentencias
}
```

</td></tr>
</table>

La sentencia while

La estructura repetitiva `while` ejecuta el bloque de sentencias si la condición contiene un valor de `true`. En caso de que el valor sea `false`, se sale del bucle.

```
while(condicion)
{
   // bloque de sentencias
}
```

```
let x = 1;
while (x <= 10) {
   document.write(x);
   x++;
}
```

La sentencia do..while

La estructura repetitiva `do-while` nos permite ejecutar al menos una vez el bloque de sentencias, ya que la condición será evaluada al finalizar el bloque.

En el caso de que la condición sea `true` se iterará nuevamente en el bucle. Si la condición es `false` se saldrá de la ejecución del bucle.

```
do {
  // Bloque de sentencias
}
while (condición);
```

```
let x = 1;
do {
    document.write(x);
    x++;
   }
while (x <= 10);
```

Siempre deberemos de asegurarnos que dentro del bloque de sentencias hay una sentencia que pueda alterar la condición y finalizar la ejecución del bucle para evitar así los dañinos bucles infinitos.

Sentencia break

Hemos visto cómo los bucles tienen una condición de salida, que una vez que se cumpla, será en este momento cuando salte a la siguiente sentencia después del bucle.

Existen otras formas de salir del bucle que no son por el cumplimiento de la condición de salida. Una de ellas es la sentencia `break` que podemos utilizarla dentro de: `for, while, do-while o switch.`

```
let x = 1;
while (x <= 10) {
if (x == 5) {
  break;
}
document.write(x);
x++;
}
```

Números aleatorios

Los números aleatorios son importantísimos en múltiples ámbitos del mundo de la programación. La función de Javascript que nos permite generar un número aleatorio es `Math.random()`:

```
var azar=Math.random();
       alert("Ha salido el: "+azar);
```

Esta sentencia almacena en la variable `azar` un número aleatorio comprendido entre el 0 (incluido) y el 1 (excluido). Es decir, el número generado más alto sería el 0,99999999999999999...

Simular tirada de dado:
`var dado = Math.floor((Math.random() * 6) + 1);`
La expresión anterior genera un número entre el 1 y el 6. Al multiplicar por 6 el número generado por random obtenemos un número aleatorio entre el 0 y el 5,9999999999... , al sumarle 1, obtendremos algo entre el 1 y el 6,99999999999. Con la función `floor`,conseguiremos quedarnos con la parte entera del número resultante obteniendo finalmente un número entre el 1 y el 6.

Siguiendo este método, si quisiéramos obtener un número entre el 0 y el 3 haríamos:

```
var num=Math.floor((Math.random() * 4);
```

Si lo que queremos ahora es obtener un número aleatorio entre otros dos, por ejemplo entre 7 y 75:

el método a seguir es multiplicar por la diferencia entre el mayor y el menor más uno:

```
Math.random() *((75-7)+1)
```

sumarle al resultado el menor:

```
(Math.random() *69)+7
```

y truncar la parte decimal del resultado:

```
var num=Math.floor(((Math.random() * 69)+7);
```

Ventanas de diálogo con el usuario

Javascript dispone de varias funciones que nos permiten notificar, advertir o pedir información al usuario:

- alert
- confirm
- prompt

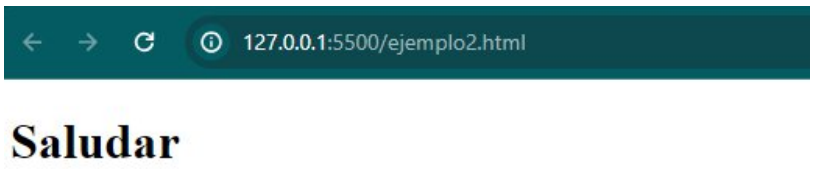
alert

La función `alert` nos muestra una ventana con un texto y un botón de Aceptar cuya única función es cerrar el cuadro de diálogo y continuar con el contenido de la página.

```
<!DOCTYPE html>
<html>
<head>
    <title>Hola Mundo Javascript</title>
</head>
<body>
    <script>
        alert("Hola mundo");
    </script>
    <h1>Saludar</h1>
</body>
</html>
```

En el navegador veremos:

Una vez pulsemos “Aceptar”, la ejecución de la página continuará y veremos:



confirm

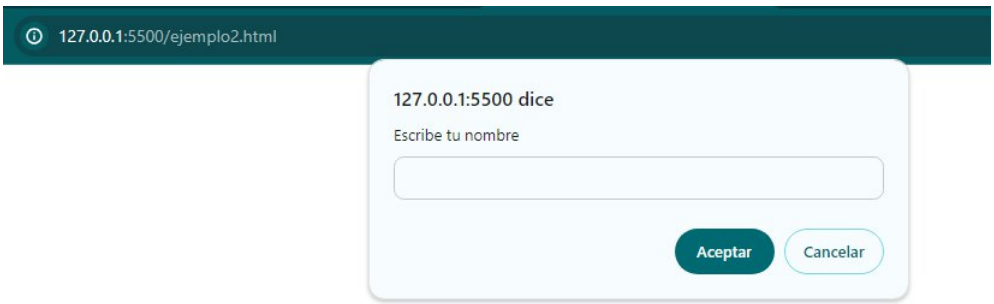
La ventana `confirm` muestra dos botones (Aceptar y Cancelar) y devuelve la elección del usuario para que podamos actuar en consecuencia como una variable booleana: true, si hemos pulsado Aceptar y false, si hemos pulsado Cancelar:

```
<script>
    var confirmacion = confirm("Pulsa el botón que quieras");
    if (confirmacion) {
        alert("Has pulsado aceptar");
    } else {
        alert("Has pulsado cancelar");
    }
</script>
```

prompt

La ventana `prompt` nos muestra una caja de texto y devuelve el valor que hemos escrito en ella. Además, muestra dos botones y devuelve el valor null (false) si hemos salido pulsando el botón de Cancelar en lugar del de Aceptar:

```
<script>
    var textoEscrito = prompt("Escribe tu nombre");
      if (textoEscrito == null) {
          alert("Has pulsado Cancelar");
      }
      else {
          if (textoEscrito.length != 0) {
              alert("Hola " + textoEscrito + "!");
          } else {
              alert("Hola persona sin nombre!");
          }
      }
</script>
```



Funciones

Una función es un conjunto de instrucciones que siempre se ejecutan unidas y a las cuales podemos invocar en diferentes partes de nuestro programa

Javascript trae muchas funciones definidas y ya hemos visto algunas de ellas (alert, prompt, etc..). Pero también, nos permite crear nuestras propias funciones.

Para definir funciones en Javascript debemos seguir la siguiente estructura:

```
function nombre_funcion(argumento1,argumento2,...)
{
  Sentencias;
  return valor_retorno:
}
```

Las llaves siempre son obligatorias.

Ejemplos

```
function saludo()
    {
       alert("Hola mundo");
    }
```

Esta función de nombre `saludo`, no recibe ningún argumento ni devuelve ningún valor, lo único que hace es mostrar una ventana con el texto "Hola mundo"

```
function mostrar_texto(texto)
    {
       alert(texto);
    }
```

Esta función de nombre `mostrar_texto`, recibe el argumento `texto` y lo muestra en una ventana

```
function mostrar_texto_y_tamanyo(texto)
    {
       alert(texto);
       return texto.length;
    }
```

Esta función de nombre `mostrar_texto_y_tamanyo`, recibe el argumento `texto`, lo muestra en una ventana y devuelve su longitud.

Para ejecutar la función basta con poner su nombre

```
saludo();

mostrar_texto("hola");

var tam=mostrar_texto_y_tamanyo("hola");
```

No se puede llamar a una función si esta no ha sido declarada aún, por eso es buena idea poner siempre las funciones en el `head` del html y llamarlas desde el body.

Otra opción aún mejor es extraer las funciones a un fichero externo distinto al html desde donde se las llama. De esta forma, además, podremos reutilizarlas desde distintas páginas sin tener que duplicar el código.

Para incluir un fichero externo con funciones javascript en nuestro html incluimos la siguiente línea en el head de la página:

```
<script type="text/javascript" src="funciones.js"></script>
```

El fichero `funciones.js` incluirá las definiciones de las funciones sin necesidad de usar ninguna etiqueta script ni de otro tipo. Como valor del argumento `src` hay que expresar una ubicación absoluta o relativa pero completa y válida para el archivo que contiene las funciones.

Arrays

Los arrays o vectores son un tipo especial de variables que siempre hay que declarar:

```
var notas_alumnos = new Array(30);
var frutas=["Manzana","Pera","Kiwi"];
```

Podemos imaginar un vector como un conjunto de valores homogéneos y que referenciamos con el mismo nombre acompañado de un índice o número de orden. Por ejemplo, el vector anterior valdría para almacenar las notas de los 30 alumnos de una clase. Para acceder a un determinado elemento del vector usamos corchetes e indicamos dentro el número de orden. El primer elemento siempre tiene el número de orden 0.

`var nota1=notas_alumnos[0];`	Obtenemos la primera nota del array

`if(notas_alumnos[8]<5)`
`{`
`alert("el alumno 9 ha suspendido");`
`}`
	Trabajamos con el elemento que ocupa la posición 8 del array

`var tam=notas_alumnos.length;`	La propiedad `length`, nos proporciona la longitud del array, es decir, el número de elementos que puede contener. No se puede modificar el valor de la propiedad.

Cadenas de texto (Strings)

Las cadenas de texto en Javascript son un tipo de array en el que cada posición del array es uno de los caracteres de la cadena de texto:

```
var cadena="Soy una cadena de texto";
var cadena='Yo también soy una cadena de texto';
```

Las cadenas de texto se pueden especificar usando comillas simples o dobles.

Una de las propiedades más utilizadas es la propiedad `length` que proporciona la longitud de la cadena:

```
var cadena = "HOLA";

alert("La longitud de la cadena es de " + cadena.length + " caracteres");
```

Las cadenas de texto aunque sean arrays, no se pueden modificar directamente:

```
var cadena = "HOLA";

cadena[1]=0;
```

Aunque este código es correcto sintácticamente y debería dar como resultado `H0LA` , la cadena original no se habrá modificado ya que está protegida contra modificaciones. Si queremos cambiar la cadena, deberemos utilizar funciones auxiliares que veremos más adelante.

Javascript tiene muchos métodos y funciones auxiliares para manipular cadenas, a continuación vemos algunas de ellas:

Concatenar cadenas de texto

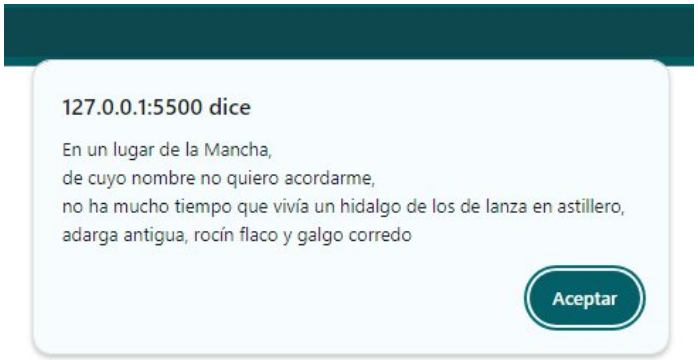
Si queremos concatenar cadenas de text podemos recurrir al operador +

```
var nombre=”Benito”;

var saludo=”Hola “+nombre;
```

Cadenas de texto multilínea

En el caso de que estemos manejando cadenas de texto muy largas podemos codificarlas como cadenas de texto multilínea. Es decir, podremos distribuir su contenido en varias líneas.

```
let cadena = "En un lugar de la Mancha, \n" +

"de cuyo nombre no quiero acordarme, \n" +

"no ha mucho tiempo que vivía " +

"un hidalgo de los de lanza en astillero, " +

"adarga antigua, rocín flaco y galgo corredo";

alert(cadena);
```



La propiedad charAt(i)

La propiedad `charAt(i)` devuelve el carácter situado en la posición i de una determinada cadena de texto. Por ejemplo, para preguntar si la posición 0 de una cadena llamada texto es un espacio haríamos esto:

```
if(texto.charAt(0) == " ")

alert("La cadena empieza por un espacio en blanco");
```

Los métodos toUpperCase() y toLowerCase()

Convierten a mayúsculas o minúsculas el contenido de una cadena:

```
texto = prompt("Escribe un texto");

mayusculas = texto.toUpperCase()

document.write("Tu texto escrito en mayúsculas es: " + mayusculas);
```